TALENT GRAPH

Bug Fix Architecture & Resolution Guide

19 Issues · Step-by-Step Fixes · Full Code Architecture

Firebase Firestore + Next.js + Africa's Talking SMS

Category	Issues	Priority
Bugs & Broken Features	7	Critical
UX & Homepage Cleanup	6	High
Data Sync Logic	2	High
Analyst Network Feedback	4	High
TOTAL	19	—

Stanley Omenda · Verve Vigor Limited · May 2026 · Confidential

TABLE OF CONTENTS

SECTION 01 | 7 Issues — All Critical

Bugs & Broken Features

BUG #01 | MODULE: CLUB — CHAT

Club Chat Isolated from Athletes, Coaches & Other Users

PRIORITY: CRITICAL	STATUS: OPEN	STACK: Firebase + Next.js

PROBLEM

Club chat is isolated to the club dashboard only. Athletes, coaches and analysts cannot access it. The chat exists but is disconnected from every other role on the platform.

ROOT CAUSE

- Chat component rendered only inside ClubDashboard layout
- No chat access route on athlete, coach or analyst dashboards
- chatMembers collection not populated on club join events
- No real-time listener outside the club dashboard context

ARCHITECTURE

Layer	What to Build
Firestore	clubChats/{clubId}/messages + chatMembers sub-collections
Cloud Function	Auto-add user to chatMembers when they join a club or squad
Cloud Function	Auto-set left_at when user leaves club or squad
Routes	Add /athlete/club-chat, /coach/club-chat, /analyst/club-chat
Real-time	Firestore onSnapshot listener shared across all dashboard layouts

STEP-BY-STEP FIX

STEP 1 — Create Firestore Schema

- clubChats/{clubId} — one document per club, created when club registers
- clubChats/{clubId}/messages/{msgId} — each chat message
- clubChats/{clubId}/members/{userId} — membership record per user

```
// clubChats/{clubId}/members/{userId}
{
  userId,
  role: "athlete" | "coach" | "analyst" | "admin",
  joinedAt: serverTimestamp(),
  leftAt: null
}

// clubChats/{clubId}/messages/{msgId}
{
```

```
  senderId, senderName, senderRole,
  body, sentAt: serverTimestamp(),
  isPinned: false, isDeleted: false,
  readBy: []
}
```

STEP 2 — Write Cloud Functions to Auto-Add Members

- Trigger 1: when athlete joins a squad (squadMembers write)
- Trigger 2: when coach is linked to a club (clubs update)
- Trigger 3: when analyst is linked to a club
- Trigger 4: on leave/removal — set leftAt timestamp

```
// functions/index.js
exports.onSquadMemberAdded = functions.firestore
  .document("squads/{coachId}/athletes/{athleteId}")
  .onCreate(async (snap, ctx) => {
    const { athleteId } = ctx.params
    const coachDoc = await getDoc(doc(db,"users",ctx.params.coachId))
    const clubId = coachDoc.data().clubId
    if (!clubId) return
    await setDoc(
      doc(db,"clubChats",clubId,"members",athleteId),
      { userId: athleteId, role: "athlete",
        joinedAt: serverTimestamp(), leftAt: null }
    )
  })
```

STEP 3 — Build Shared Chat Component

- Create one reusable ChatWindow component — not dashboard-specific
- Props: clubId, currentUser, userRole
- Uses onSnapshot to listen for new messages in real time
- Works identically on athlete, coach, analyst and club admin dashboards

```
// components/ClubChat.jsx
function ClubChat({ clubId, currentUser }) {
  const [messages, setMessages] = useState([])

  useEffect(() => {
    const q = query(
      collection(db,"clubChats",clubId,"messages"),
      orderBy("sentAt","asc"), limitToLast(50)
    )
    return onSnapshot(q, snap => {
      setMessages(snap.docs.map(d => ({ id:d.id, ...d.data() })))
    })
  }, [clubId])

  async function sendMessage(body) {
    await addDoc(
      collection(db,"clubChats",clubId,"messages"),
      { senderId: currentUser.uid,
        senderName: currentUser.name,
        senderRole: currentUser.role,
        body, sentAt: serverTimestamp(),
```

```
        isPinned: false, isDeleted: false }
    )
  }
  // render messages + input...
}
```

STEP 4 — Add Chat Route to Each Dashboard

- Athlete dashboard: /athlete/club-chat — visible under "My Club" tab
- Coach dashboard: /coach/communications — new "Club Chat" tab
- Analyst dashboard: /analyst/club-chat
- Club Admin: existing location stays, now uses same shared component
- Show unread badge count on sidebar icon for all roles

STEP 5 — Firestore Security Rules

- Only chatMembers with leftAt == null can read/write messages
- Only club admin can pin or delete other users messages
- No scout access to any clubChats path

```
// firestore.rules
match /clubChats/{clubId}/messages/{msgId} {
  allow read, create: if isChatMember(clubId, request.auth.uid);
  allow delete: if isClubAdmin(clubId, request.auth.uid);
}
function isChatMember(clubId, uid) {
  return exists(/databases/$(database)/documents/
    clubChats/$(clubId)/members/$(uid))
    && get(...).data.leftAt == null;
}
```

FALLBACK: If this is too complex before launch — remove club chat entirely from all views and defer to v2. Never ship a broken chat.

BUG #02 | MODULE: CLUB — DM

Club DM Not Functional — No Link Between Admin and Users

PRIORITY: CRITICAL	STATUS: OPEN	STACK: Firebase + Next.js

PROBLEM

Direct messaging between club admin and coaches/athletes has no working connection. The UI exists but the link is broken.

RECOMMENDED APPROACH

Defer full DM to v2. Ship a working interim solution now.

STEP-BY-STEP FIX (Interim — Ship Now)

STEP 1 — Remove Broken DM Link

- Remove any DM button that currently links to a broken route
- Do not show broken functionality to users

STEP 2 — Replace with Club Chat

- Club admin communicates with coaches and athletes via Club Chat (Bug 01 fix)
- Add a banner where DM was: "Use Club Chat to message your squad and staff"
- Direct users to the Club Chat route

STEP 3 — Full DM Build — v2 Architecture

- conversations/{conversationId} — one per pair of users
- conversations/{id}/messages/{msgId} — message thread
- Club admin can start a DM with any coach or athlete in their club
- Route: /club/messages → list → open thread

```
// v2 — conversations/{conversationId}
{
  participants: [userId1, userId2],
  clubId,
  initiatedBy,
  initiatedAt: serverTimestamp(),
  lastMessageAt,
  lastMessagePreview
}

// conversations/{id}/messages/{msgId}
{
  senderId, body,
  sentAt: serverTimestamp(),
  readAt: null
}
```

BUG #03 | MODULE: CLUB — PROFILE

Club Profile Update Throws Error on Save

PRIORITY: CRITICAL	STATUS: OPEN	STACK: Firebase + Next.js

PROBLEM

When a club admin updates their club profile, the save operation throws an error. The update does not persist.

LIKELY ROOT CAUSES

Cause	How to Confirm	Fix
Firestore rules blocking write	Check Firebase console Rules tab for denied writes	Add write rule for club admin on clubs/{clubId}
Undefined field in update payload	Check browser console for "undefined" errors	Sanitise all fields before writing
Wrong document path	Log the clubId being used in the update	Confirm clubId from auth session matches document
Missing serverTimestamp import	Check if serverTimestamp() throws	Import from firebase/firestore correctly

STEP-BY-STEP FIX

STEP 1 — Fix Firestore Security Rules

- Confirm the rule allows club admin to write to clubs/{clubId}

```
// firestore.rules
match /clubs/{clubId} {
  allow read: if request.auth != null;
  allow write: if request.auth.uid ==
    resource.data.adminId;
}
```

STEP 2 — Sanitise the Update Payload

- Remove all undefined or null fields before writing to Firestore
- Firestore rejects documents with undefined values

```
function sanitise(obj) {
  return Object.fromEntries(
    Object.entries(obj).filter(([_,v]) => v !== undefined)
  )
}

async function updateClubProfile(clubId, formData) {
  try {
    await updateDoc(doc(db,"clubs",clubId), {
      ...sanitise(formData),
      updatedAt: serverTimestamp()
    })
    showToast("Club profile saved", "success")
  } catch (err) {
    console.error("Club update error:", err.code, err.message)
    showToast("Update failed: " + err.message, "error")
  }
}
```

STEP 3 — Add User Feedback

- Success toast: "Club profile saved successfully"
- Error toast: "Update failed. Check your connection and try again."
- Show loading spinner on the Save button while request is in flight
- Disable Save button during request to prevent double-submit

---

BUG #04 | MODULE: MATCHES

Match Creation Fails But Squad Notification Still Sends

PRIORITY: CRITICAL	STATUS: OPEN	STACK: Firebase + Next.js

PROBLEM

Match creation throws an error, but despite the failure, squad athletes still receive a match notification. Error state is inconsistent and confusing to users.

ROOT CAUSE

The notification function is being called outside the try block or before awaiting the match document write. It fires regardless of whether the match was created.

STEP-BY-STEP FIX

STEP 1 — Wrap Everything in a Single Atomic Try Block

- Match document creation must fully resolve before any notification fires
- If the match write throws — catch block runs, zero notifications sent

```
async function createMatch(matchData, squadIds) {
  try {
    // Step 1: Create match — must succeed first
    const matchRef = await addDoc(
      collection(db,"matches"),
      { ...matchData, createdAt: serverTimestamp(),
        status: "scheduled" }
    )
    // Step 2: Only reached if Step 1 succeeded
    await sendSquadNotification(squadIds, {
      type: "match_created",
      matchId: matchRef.id,
      title: `Match vs ${matchData.opponent}`,
      body: `${matchData.date} at ${matchData.venue}`
    })
    showToast("Match created. Squad notified.", "success")

  } catch (err) {
    // Notification is NEVER reached here
    console.error("Match creation failed:", err)
    showToast("Match creation failed. No notification sent.", "error")
  }
}
```

STEP 2 — Fix the Notification Function

- notifications written to Firestore notifications/{userId}/items
- SMS via Africa's Talking only fires inside the success path

```
async function sendSquadNotification(squadIds, payload) {
  const batch = writeBatch(db)
  for (const uid of squadIds) {
    const ref = doc(collection(db,`notifications/${uid}/items`))
    batch.set(ref, {
      ...payload,
      read: false,
      createdAt: serverTimestamp()
    })
  }
  await batch.commit()
  // SMS only if match confirmed
  if (payload.type === "match_created") {
    await sendMatchSMS(squadIds, payload)
  }
}
```

STEP 3 — Test Both Paths

- Test success path: match creates, notification fires, toast shows
- Test failure path: disconnect Firestore, confirm NO notification fires
- Check Firestore console to verify no orphan notifications exist

---

BUG #05 | MODULE: MATCHES

Match Review Cannot Be Submitted Post-Match

PRIORITY: CRITICAL | STATUS: OPEN | STACK: Firebase + Next.js

PROBLEM

After a match is completed, the match review form fails to submit and save. Coaches cannot log post-match assessments.

STEP-BY-STEP FIX

STEP 1 — Define the Correct Firestore Path

- Match reviews are a sub-collection of the match document
- Path: matches/{matchId}/reviews/{reviewId}
- Do not write reviews as a field on the match document

STEP 2 — Fix the Write Function

- Write review as a new document in the sub-collection
- After review saves, update the parent match reviewStatus field

```
async function submitMatchReview(matchId, reviewData) {
  try {
    // Write review document
    await addDoc(
      collection(db,"matches",matchId,"reviews"),
      {
        ...reviewData,
        submittedBy: auth.currentUser.uid,
        submittedAt: serverTimestamp(),
        status: "submitted"
      }
    )
    // Update parent match status
    await updateDoc(doc(db,"matches",matchId), {
      reviewStatus: "reviewed",
      reviewedAt: serverTimestamp()
    })
    showToast("Match review saved", "success")
  } catch (err) {
    console.error("Review error:", err)
    showToast("Failed to save review. Try again.", "error")
  }
}
```

STEP 3 — Fix Firestore Rules for Sub-Collection

- Confirm the rule allows coach to write to matches/{matchId}/reviews
- The coach must be associated with the club that owns the match

```
match /matches/{matchId}/reviews/{reviewId} {
  allow create: if isCoachOfMatch(matchId, request.auth.uid);
  allow read: if isClubMember(request.auth.uid);
}
```

STEP 4 — Fix the Review Form UI

- Show loading state on submit button

- Disable submit after first click to prevent double-submit
- Navigate to match detail page on success
- Show inline error message if submission fails

BUG #06 | MODULE: MATCHES

Live Match Data Not Appending During Active Match

PRIORITY: CRITICAL	STATUS: OPEN	STACK: Firebase + Next.js

PROBLEM

During a live match, goals, cards and substitutions fail to append in real time. The live match tracking feature is non-functional.

ROOT CAUSE

Live events are likely being written as array updates on the match document. Firestore array union under concurrent writes is unreliable. Each event must be a separate document.

STEP-BY-STEP FIX

STEP 1 — Redesign the Data Model — Events Sub-Collection

- Each live event is a separate document in matches/{matchId}/events
- Do NOT use array fields on the match document for live events
- This enables concurrent writes without conflicts

```
// matches/{matchId}/events/{eventId}
{
  type: "goal"|"yellow_card"|"red_card"|"substitution"|"kickoff"|"fulltime",
  minute: 67,
  athleteId: "...",
  athleteName: "James Otieno",
  note: "",
  createdAt: serverTimestamp()
}
```

STEP 2 — Fix the Write Function

- Each event appended as addDoc to the events sub-collection
- Never use arrayUnion for live match data

```
async function appendMatchEvent(matchId, event) {
  try {
    await addDoc(
      collection(db,"matches",matchId,"events"),
      { ...event, createdAt: serverTimestamp() }
    )
  } catch (err) {
    console.error("Event append error:", err)
    showToast("Failed to save event. Retry.", "error")
  }
}
```

STEP 3 — Fix the Real-Time Read

- Use onSnapshot on the events sub-collection

- Order by minute ascending
- New events appear instantly without page refresh

```
function subscribeToMatchEvents(matchId, onUpdate) {
  const q = query(
    collection(db,"matches",matchId,"events"),
    orderBy("minute","asc")
  )
  return onSnapshot(q, snap => {
    const events = snap.docs.map(d => ({ id:d.id, ...d.data() }))
    onUpdate(events)
  })
}
// Call on match page mount, unsubscribe on unmount
```

STEP 4 — Test

- Open match on two devices simultaneously
- Add event on device 1 — confirm it appears on device 2 within 1 second
- Test each event type: goal, card, substitution

BUG #07 | MODULE: PASSWORD RESET

Password Reset Flow Completely Non-Functional

PRIORITY: CRITICAL	STATUS: OPEN	STACK: Firebase + Next.js

PROBLEM

Users cannot reset their password. The reset flow is broken end to end. Users who forget their password cannot log in.

STEP-BY-STEP FIX

STEP 1 — Check Firebase Console Configuration

- Go to Firebase Console → Authentication → Templates → Password Reset
- Confirm the email template is set up and not blank
- Go to Authentication → Settings → Authorized Domains
- Add talentgraph.co.ke and talent-graph.vercel.app to authorized domains
- If domains are missing, reset emails will be blocked silently

STEP 2 — Implement sendPasswordResetEmail

- Use Firebase Auth built-in function
- Point the continue URL to your login page

```
import { sendPasswordResetEmail } from "firebase/auth"

async function handlePasswordReset(email) {
  try {
    await sendPasswordResetEmail(auth, email, {
      url: "https://talentgraph.co.ke/login",
      handleCodeInApp: false
    })
    // Do NOT reveal if email exists or not
    showToast("If this email exists, a reset link has been sent.", "success")
  } catch (err) {
```

```
    if (err.code === "auth/invalid-email") {
      showToast("Please enter a valid email address.", "error")
    } else {
      showToast("If this email exists, a reset link has been sent.", "success")
    }
  }
}
```

STEP 3 — Build the Reset Landing Page (/reset-password)

- Firebase sends the user a link with an oobCode in the URL
- Your app must handle this oobCode to complete the reset

```
import { confirmPasswordReset, verifyPasswordResetCode } from "firebase/auth"

async function confirmNewPassword(oobCode, newPassword) {
  try {
    // Verify the code is still valid
    await verifyPasswordResetCode(auth, oobCode)
    // Confirm and set new password
    await confirmPasswordReset(auth, oobCode, newPassword)
    showToast("Password updated. You can now log in.", "success")
    router.push("/login")
  } catch (err) {
    if (err.code === "auth/expired-action-code") {
      showToast("Reset link has expired. Please request a new one.", "error")
    } else {
      showToast("Invalid link. Please request a new reset link.", "error")
    }
  }
}

// On page load — extract oobCode from URL
const params = new URLSearchParams(window.location.search)
const oobCode = params.get("oobCode")
```

STEP 4 — Password Validation on the Form

- Minimum 8 characters
- Confirm password field must match
- Show strength indicator
- Disable submit until both fields are valid

STEP 5 — Test End to End

- Request reset for a real test account email
- Confirm email arrives (check spam)
- Click link, confirm landing page loads with form
- Set new password, confirm redirect to login
- Log in with new password — confirm success

SECTION 02 | 6 Issues

UX & Homepage Cleanup

BUG #08 | MODULE: HOMEPAGE

Scouting Pipeline Redundant on Homepage

PRIORITY: HIGH	STATUS: OPEN	STACK: Firebase + Next.js

PROBLEM

Scouting pipeline appears on homepage but is already accessible via the (+) button and More menu. Duplicate entry point creates confusion.

STEP-BY-STEP FIX

STEP 1 — Remove from Homepage

- Locate the scouting pipeline card/widget in the homepage component
- Remove it entirely — no replacement needed on the homepage
- Do not add any shortcut or placeholder in its place

STEP 2 — Confirm Existing Access Points Still Work

- Test: (+) button still opens scouting pipeline — confirm yes
- Test: More menu still shows scouting pipeline link — confirm yes
- If either is broken, fix the existing access point rather than keeping the homepage duplicate

STEP 3 — Clean the Homepage Layout

- After removal, check the homepage layout for gaps or spacing issues
- Re-order remaining cards if needed for visual balance
- Test on 360px, 390px and 414px screen widths

BUG #09 | MODULE: HOMEPAGE — SCOUT REQUEST

Messaging Button Clutters Scout Request Card

PRIORITY: HIGH	STATUS: OPEN	STACK: Firebase + Next.js

PROBLEM

After a scout request is accepted, a message button appears directly on the request card face. This clutters the card and makes it feel crowded.

STEP-BY-STEP FIX

STEP 1 — Remove Messaging Button from Card Face

- Delete the Message button from the accepted request card component
- Card face should only show: photo, name, status badge, timestamp

STEP 2 — Add Three-Dot Menu

- Add a three-dot (vertical ellipsis) icon button to the top-right of the card

- On tap, open a bottom sheet or dropdown menu with three options:
- — Message (navigates to messaging page with this thread open)
- — Disconnect (removes the connection with confirmation modal)
- — Report (flags this connection for admin review)

```
// ScoutRequestCard.jsx
function ScoutRequestCard({ request }) {
  const [menuOpen, setMenuOpen] = useState(false)
  return (
    <div className="card">
      <Avatar src={request.photo} />
      <div>
        <p>{request.name}</p>
        <StatusBadge status={request.status} />
        <span>{timeAgo(request.createdAt)}</span>
      </div>
      <ThreeDotButton onClick={() => setMenuOpen(true)} />
      {menuOpen && (
        <DropdownMenu>
          <MenuItem onClick={() => router.push(`/messages/${request.id}`)}>
            Message
          </MenuItem>
          <MenuItem onClick={handleDisconnect}>Disconnect</MenuItem>
          <MenuItem onClick={handleReport}>Report</MenuItem>
        </DropdownMenu>
      )}
    </div>
  )
}
```

STEP 3 — Ensure Message Route Pre-Opens Correct Thread

- When navigating to /messages from this menu, pass the conversationId
- Messaging page should auto-open that thread, not the inbox list
- Route: /messages?conversation={request.id}

BUG #10 | MODULE: HOMEPAGE — NOTIFICATIONS

Notification Card Redundant on Homepage

PRIORITY: HIGH	STATUS: OPEN	STACK: Firebase + Next.js

PROBLEM

A notification card on the homepage duplicates the dedicated Alerts page. Redundant and creates visual clutter.

STEP-BY-STEP FIX

STEP 1 — Remove Notification Card from Homepage

- Delete the notification card/widget from the homepage layout
- No replacement on the homepage

STEP 2 — Add Bell Icon to Navigation Bar

- Add a bell icon to the top navigation bar (consistent across all dashboards)

- Show an unread count badge on the bell when notifications exist
- Tapping the bell navigates to /alerts (the existing Alerts page)

```
// NavBar.jsx
function NavBar() {
  const { unreadCount } = useNotifications()
  return (
    <nav>
      {/* other nav items */}
      <button onClick={() => router.push("/alerts")}
        className="relative">
        <BellIcon />
        {unreadCount > 0 && (
          <span className="badge">{unreadCount}</span>
        )}
      </button>
    </nav>
  )
}

// useNotifications hook
function useNotifications() {
  const [unreadCount, setUnreadCount] = useState(0)
  useEffect(() => {
    const q = query(
      collection(db,`notifications/${uid}/items`),
      where("read","==",false)
    )
    return onSnapshot(q, snap => setUnreadCount(snap.size))
  }, [])
  return { unreadCount }
}
```

STEP 3 — Add Delete Button to Individual Notifications on Alerts Page

- Each notification row on the Alerts page gets a delete (X) button
- On delete: remove the Firestore document from notifications/{uid}/items
- Also add "Clear All" button at the top of the Alerts page

BUG #11 | MODULE: HOMEPAGE — TALENT MARKETPLACE

Talent Marketplace Shortcut Redundant on Homepage

PRIORITY: MEDIUM	STATUS: OPEN	STACK: Firebase + Next.js

PROBLEM

Talent marketplace shortcut visible on homepage but is already accessible via settings.

STEP-BY-STEP FIX

STEP 1 — Remove Shortcut from Homepage

- Delete the talent marketplace card/shortcut from the homepage layout

STEP 2 — Confirm Existing Access Points

- Settings menu: confirm marketplace link works

- Scout dashboard sidebar: confirm marketplace navigation works
- No other changes needed

STEP 3 — Homepage Guiding Principle

- Homepage should only show: daily priority actions, urgent notifications, performance snapshot
- Feature shortcuts that have their own navigation home elsewhere must not appear on homepage
- Apply this rule to any future homepage additions

BUG #12 | MODULE: PROFILE — ALTERNATE POSITION

Alternate Position Tap Navigates to CSI Index (Wrong)

PRIORITY: HIGH	STATUS: OPEN	STACK: Firebase + Next.js

PROBLEM

On the profile strength card, tapping the position or alternate position field incorrectly navigates to the CSI index page. This is the wrong destination.

STEP-BY-STEP FIX

STEP 1 — Find and Fix the Wrong onPress Handler

- Locate the position field tap handler in the profile strength card component
- Change the navigation target from CSI index to the edit profile screen

```
// WRONG — current behaviour
onPress={() => router.push("/csi-index")}

// CORRECT — fixed behaviour
onPress={() => router.push("/profile/edit#positions")}
// OR open a bottom sheet modal:
onPress={() => setPositionModalOpen(true)}
```

STEP 2 — Build the Position Edit Section

- In Edit Profile, create a dedicated "Positions" section
- Primary position: dropdown selector (GK, CB, LB, RB, CDM, CM, CAM, LW, RW, ST)
- Alternate position: second dropdown (same options, "None" as default)
- Save button updates both fields in the athletes Firestore document

STEP 3 — Anchor Link Navigation

- When tapping position from profile strength card, scroll to the positions section
- Use #positions anchor or scroll the edit form to that section automatically
- Confirm the user lands on the correct section, not the top of the edit form

---

BUG #13 | MODULE: CLUB TRIAL FEATURE

Club Trial Feature Has No Product Definition

PRIORITY: MEDIUM	STATUS: OPEN	STACK: Firebase + Next.js

PROBLEM

Engineers do not know what the club trial feature is supposed to do. Insufficient definition has blocked development.

FULL PRODUCT DEFINITION

STEP 1 — What the Feature Does

- Club admin pays KES 3,500 via M-Pesa STK Push to unlock a specific athlete
- On payment: athlete full contact details and verified stats revealed to this club
- 14-day trial window opens from payment timestamp
- Athlete is notified that a club has unlocked their profile for a trial

STEP 2 — Build the Firestore Schema

- trialUnlocks/{unlockId} — one document per unlock transaction

```
// trialUnlocks/{unlockId}
{
  clubId,
  athleteId,
  amountKES: 3500,
  status: "payment_pending"|"active"|"expired"|"converted",
  checkoutRequestId,  // M-Pesa STK Push reference
  unlockedAt,
  trialWindowEnd,     // unlockedAt + 14 days
  trialOutcome: null, // "signed"|"not_signed"|"ongoing"
  createdAt: serverTimestamp()
}
```

STEP 3 — Build the Unlock Flow

- Club admin finds athlete, clicks "Unlock for Trial" button
- Modal shows: athlete name, cost (KES 3,500), M-Pesa number input
- On confirm: trigger M-Pesa STK Push, show "Check your phone" spinner
- On payment success (M-Pesa callback): set status to "active"
- Grant club access to athlete contact details and full verified stats
- Send SMS to athlete: "A club has unlocked your profile for a trial. Login to view."
- Send SMS to club admin: "Profile unlocked. KES 3,500 received. 14-day window open."

STEP 4 — Build Trial Outcome Logging

- After 14 days: show club admin a prompt "Log trial outcome"
- Options: Signed / Not Signed / Extend (requires another payment)
- Outcome saved to trialUnlocks document
- If Signed: create a transfer record linking athlete to club
- NOTE: Do NOT build insurance logic yet. Insurance language requires a signed underwriter agreement first.

SECTION 03 | 2 Issues

Data Sync Logic

BUG #14 | MODULE: DATA SYNC — ATHLETE/CLUB

Athlete Game Data Exists in Isolation — Not Synced with Club

PRIORITY: HIGH	STATUS: OPEN	STACK: Firebase + Next.js

PROBLEM

A game logged by an athlete exists as a standalone document with no link to club data. It does not appear in the club's match records and does not cross-reference from the date the athlete joined.

ROOT CAUSE

matchStats documents are written with only athleteId. The clubId field is missing or not being set, so club queries cannot find them.

ARCHITECTURE — SINGLE SOURCE OF TRUTH

Every matchStats document must contain BOTH athleteId and clubId. The same document is returned by both an athlete query and a club query.

STEP-BY-STEP FIX

STEP 1 — Fix the matchStats Schema

- Add clubId to every matchStats write from now on
- Run a migration to backfill clubId on existing documents where the athlete has a club

```
// matchStats/{matchId} — complete schema
{
  athleteId,
  clubId,           // REQUIRED — even if self-reported
  coachId,          // null if athlete entered
  enteredBy: "athlete"|"coach"|"club",
  verificationStatus: "self_reported"|"coach_verified",
  season,
  competition,
  date,
  opponent,
  // ...all stat fields
  createdAt: serverTimestamp()
}
```

STEP 2 — Fix the Write Functions

- When athlete logs a match: fetch their current clubId from their user document, include it
- When coach logs a match: always include clubId from the coach's club

```
// Athlete self-reporting a match
async function athleteLogMatch(athleteId, matchData) {
  const userDoc = await getDoc(doc(db,"users",athleteId))
  const clubId = userDoc.data().clubId || null
```

```
  await addDoc(collection(db,"matchStats"), {
    ...matchData,
    athleteId,
    clubId,           // include even if null
    enteredBy: "athlete",
    verificationStatus: "self_reported",
    createdAt: serverTimestamp()
  })
}
```

STEP 3 — Fix the Query Functions

- Athlete profile reads: filter by athleteId only
- Club dashboard reads: filter by clubId only
- Both queries return the same documents when clubId is set

```
// Athlete personal match history
query(collection(db,"matchStats"),
  where("athleteId","==",athleteId),
  orderBy("date","desc"))

// Club match records
query(collection(db,"matchStats"),
  where("clubId","==",clubId),
  orderBy("date","desc"))

// Cross-reference from join date
query(collection(db,"matchStats"),
  where("athleteId","==",athleteId),
  where("clubId","==",clubId),
  where("date",">=",athlete.clubJoinDate),
  orderBy("date","desc"))
```

STEP 4 — Run Backfill Migration

- Write a one-time script to update existing matchStats missing clubId
- For each matchStats doc: look up the athlete's clubId at that time
- Set clubId if found, leave null if athlete had no club
- Run in Firestore using a batch write — 500 docs per batch

BUG #15 | MODULE: DATA SYNC — CLUB/ATHLETE

Club Match Not Appearing on Player Individual Profile

PRIORITY: HIGH	STATUS: OPEN	STACK: Firebase + Next.js

PROBLEM

When a club adds a match and marks players as attending, the match data does not appear on those players individual profiles.

ROOT CAUSE

Same as Bug 14: matchStats written by club use only clubId with no athleteId, so athlete profile queries cannot find them.

STEP-BY-STEP FIX

STEP 1 — Fix Club Match Entry to Write Per-Athlete Records

- When a coach submits match data, create one matchStats document per athlete who played
- Each document has both clubId and athleteId
- Use a Firestore batch write to create all documents atomically

```
async function submitClubMatch(matchData, playerStats) {
  // playerStats = [{ athleteId, goals, assists, ... }]
  const batch = writeBatch(db)

  for (const player of playerStats) {
    const ref = doc(collection(db,"matchStats"))
    batch.set(ref, {
      ...matchData,          // date, opponent, competition
      ...player,             // athleteId + stats
      clubId: matchData.clubId,
      coachId: auth.currentUser.uid,
      enteredBy: "coach",
      verificationStatus: "coach_verified",
      createdAt: serverTimestamp()
    })
  }

  await batch.commit()
  showToast(`Match saved. ${playerStats.length} player records updated.`, "success")
}
```

STEP 2 — Fix Attendance Confirmation to Trigger Stat Write

- When a player confirms attendance for a scheduled match:
- → On match result entry, their stats are written automatically
- → No manual linking required

STEP 3 — Verify on Player Profile

- After fix: coach enters match, athlete opens their profile
- Athlete profile Performance section should show the new match
- Composite score should recalculate after the write completes
- Test with 3 different athletes to confirm all records appear

SECTION 04 | 4 Issues — via Warren / TAA

Analyst Network Feedback

BUG #16 | MODULE: PLATFORM-WIDE

Target Audience Not Clear to New Users on Homepage

PRIORITY: HIGH	STATUS: OPEN	STACK: Firebase + Next.js

PROBLEM

KPL analysts landing on the platform do not immediately understand who it is built for. The homepage does not communicate the target audience or the value proposition within the first 3 seconds.

STEP-BY-STEP FIX

STEP 1 — Redesign the Homepage Hero Section

- Replace current hero with a clear value statement:
- Headline: "Kenya's Football Intelligence Platform"
- Sub-headline: "Verified athlete data for coaches, scouts, clubs and analysts."
- This tells a new visitor exactly who the platform serves

STEP 2 — Add Four Role Entry Cards Below the Hero

- Four cards in a 2x2 grid (mobile) or row (desktop)
- Each card: icon, role name, one-line description, CTA button
- Athlete: "Build your profile. Get discovered." → Join Free
- Coach / Analyst: "Verify data. Manage your squad." → Join
- Scout: "Discover and recruit verified talent." → Join
- Club: "Manage your squad and track performance." → Join
- Each card button links directly to the role-specific onboarding flow

```
// homepage/RoleCards.jsx
const roles = [
  { icon: "■", title: "Athlete",
    desc: "Build your profile. Get discovered.",
    cta: "Join Free", href: "/signup?role=athlete" },
  { icon: "■", title: "Coach / Analyst",
    desc: "Verify data. Manage your squad.",
    cta: "Join", href: "/signup?role=coach" },
  { icon: "■", title: "Scout",
    desc: "Discover and recruit verified talent.",
    cta: "Join", href: "/signup?role=scout" },
  { icon: "■■", title: "Club",
    desc: "Manage your squad and track performance.",
    cta: "Join", href: "/signup?role=club_admin" },
]
```

STEP 3 — Pre-Select Role on Signup from Card Click

- When user clicks a role card, pass ?role=X as a query param to /signup
- Signup page reads the query param and pre-selects the correct role in the dropdown

- User does not need to re-select — reduces friction

STEP 4 — Above the Fold Rule

- All critical information (headline, sub-headline, role cards) must be visible
- without scrolling on a 390px mobile screen
- Test on real device before shipping

BUG #17 | MODULE: PLATFORM-WIDE

Platform Too Complex for First-Time Users — Analysts Hit Errors

PRIORITY: HIGH	STATUS: OPEN	STACK: Firebase + Next.js

PROBLEM

New users, especially analysts, hit errors on first use because the platform shows too many options at once with no guidance on where to start.

STEP-BY-STEP FIX

STEP 1 — Build a Role-Specific Onboarding Checklist

- Show immediately after first login for all new users
- Pinned to top of dashboard until all steps complete
- Each step is a tappable link to the exact action
- Dismissible after step 2 is completed

Role	Step 1	Step 2	Step 3	Step 4
Coach / Analyst	Complete profile	Apply for verification	Add first athlete to squad	Enter first match result
Scout	Complete profile	Run first search	Save an athlete	Send first scout request
Club Admin	Complete club profile	Add first coach	Add first athlete	View squad analytics

```
// OnboardingChecklist.jsx
function OnboardingChecklist({ role, completedSteps }) {
  const steps = ONBOARDING_STEPS[role] // per-role step config
  const progress = completedSteps.length / steps.length

  if (progress === 1 && dismissed) return null

  return (
    <div className="checklist-card">
      <p>Getting started — {completedSteps.length} of {steps.length} done</p>
      <ProgressBar value={progress} />
      {steps.map(step => (
        <ChecklistItem
          key={step.id}
          step={step}
          done={completedSteps.includes(step.id)}
          onClick={() => router.push(step.href)}
        />
      ))}
```

```
      {completedSteps.length >= 2 &&
        <button onClick={dismiss}>Dismiss</button>}
    </div>
  )
}
```

STEP 2 — Track Onboarding Completion in Firestore

- users/{userId}.onboardingCompleted = [] (array of completed step IDs)
- When user completes an action (e.g. submits first match), add step ID to array
- Checklist reads from this array to show which steps are done

STEP 3 — Simplify Navigation for New Users

- For users with 0 completed onboarding steps: grey out advanced features
- Show tooltip on greyed features: "Complete setup to unlock this"
- Do not hide features entirely — let users see what is coming

---

BUG #18 | MODULE: DATA PORTABILITY

Analyst Data Locked to Club — Cannot Move with Coach/Analyst

PRIORITY: HIGH	STATUS: OPEN	STACK: Firebase + Next.js

PROBLEM

When a coach or analyst changes clubs, their personal data (verifications, notes, reports) is locked to the old club account. They lose access to everything they built.

ARCHITECTURE — DATA OWNERSHIP RULES

Data Type	Owned By	Moves With User?	Access After Club Change
Squad roster	Club	No	Stays with club
Club match records	Club	No	Stays with club
Club statistics	Club	No	Stays with club
Verification history	Coach	Yes	Follows coach to new club
Personal athlete notes	Coach / Scout	Yes	Follows user
Generated reports	Scout	Yes	Follows scout
Saved searches	Scout	Yes	Follows scout

STEP 1 — Add dataOwner Field to All Records

- Every Firestore document must declare who owns it
- This determines what happens when the user changes clubs

```
// matchStats — owned by club
{ clubId, athleteId, dataOwner: "club" }

// verifications — owned by coach
{ coachId, athleteId, dataOwner: "coach" }

// scoutNotes — owned by scout
{ scoutId, athleteId, dataOwner: "scout" }
```

STEP 2 — Fix the Club Change Flow

- When coach/analyst updates their clubId in their user document:
- → Their personal records remain accessible (query by coachId)
- → Old club records remain with the old club (query by clubId)
- → New club cannot see old club private data
- No data migration needed — the dataOwner field handles the separation

STEP 3 — Fix the Queries to Respect Ownership

- Coach personal dashboard: query verifications by coachId (not clubId)
- Club dashboard: query records by clubId (not coachId)
- These two queries never overlap unless the coach is still at the same club

```
// Coach personal verification history
// Accessible regardless of current club
query(collection(db,"verifications"),
  where("coachId","==",coachId))

// Club match records
// Tied to club — coach cannot take these
query(collection(db,"matchStats"),
  where("clubId","==",currentClubId))
```

BUG #19 | MODULE: ADMIN FLOW

Athletes Required to Enter Their Own Data — Wrong Flow

PRIORITY: HIGH	STATUS: OPEN	STACK: Firebase + Next.js

PROBLEM

The current flow requires athletes to self-enter their own performance data. Analysts and coaches prefer to enter data themselves and then give players access to view their verified stats.

THE NEW FLOW — COACH/ANALYST FIRST

Old Flow (Wrong)	New Flow (Correct)
Athlete logs in and enters own stats	Coach/Analyst enters all match stats for squad
Stats show as Self Reported	Stats show as Coach Verified immediately
Athlete must remember to update after every match	Athlete receives notification and views their stats
Data quality is unreliable	Data quality is trusted from entry

STEP 1 — Remove Mandatory Stat Entry from Athlete Onboarding

- Current athlete onboarding asks them to enter stats as a required step
- Remove this requirement — athlete onboarding should only collect:
- → Photo upload
- → Bio / personal statement
- → Primary position and alternate position
- → Physical attributes (height, weight, dominant foot)

- These are things only the athlete knows. Match stats come from the coach.

STEP 2 — Change the Athlete Stats Section UI

- When athlete has no coach-entered data, show this message in their stats section:
- "Your performance stats will appear here once your coach enters match data.
- Ask your coach to add you to their squad on Talent Graph."
- Do not show blank stat fields waiting to be filled by the athlete

STEP 3 — Notify Athlete When Coach Enters Their Data

- After coach submits match data for the squad:
- → Each athlete in that match receives a notification
- → In-app: "Your match stats have been updated by Coach [Name]"
- → SMS: "TalentGraph: Your stats were updated by Coach [Name]. Login to view."

```
// After coach submits match — notify each athlete
async function notifyAthletesOfStatUpdate(playerStats, coachName) {
  for (const player of playerStats) {
    // In-app notification
    await addDoc(
      collection(db,`notifications/${player.athleteId}/items`),
      {
        type: "stats_updated",
        title: "Your stats have been updated",
        body: `Coach ${coachName} entered your match data.`,
        read: false,
        createdAt: serverTimestamp()
      }
    )
    // SMS
    await sendSMS(player.phone,
      `TalentGraph: Your stats were updated by Coach ${coachName}. Login to view.`
    )
  }
}
```

STEP 4 — Allow Athletes to Self-Report Only If No Coach

- If athlete has no squad affiliation: allow self-reporting with ■ Self Reported badge
- Once athlete joins a coach squad: disable self-reporting of match stats
- Coach-entered data cannot be edited by the athlete
- Athlete can still edit: bio, photo, position, physical attributes

```
// matchStats field to control edit access
{
  enteredBy: "coach",
  athleteCanEdit: false,  // locked once coach enters
}

// In athlete edit profile component
const canEditStats = matchStat.enteredBy === "athlete"
  && matchStat.athleteCanEdit !== false
```

APPENDIX

Master Fix Priority Order

Fix in this exact order. Do not start a lower priority bug until all higher priority bugs in the same phase are resolved and tested.

Phase	Bug #	Module	Priority	Estimated Effort
TODAY	07	Password Reset	Critical	2–3 hours
TODAY	03	Club Profile Update Error	Critical	2–4 hours
TODAY	04	Match Create / Notify Consistency	Critical	3–4 hours
TODAY	05	Match Review Submission	Critical	3–5 hours
TODAY	06	Live Match Data Appending	Critical	4–6 hours
THIS WEEK	01	Club Chat Access (all roles)	Critical	1–2 days
THIS WEEK	02	Club DM (defer to v2)	Critical	2 hours (defer)
THIS WEEK	14	Data Sync Athlete/Club	High	4–6 hours + migration
THIS WEEK	15	Club Match on Player Profile	High	3–4 hours
NEXT WEEK	16	Homepage Clarity	High	1 day
NEXT WEEK	17	Onboarding Flow	High	2–3 days
NEXT WEEK	08	Homepage Scouting Pipeline	High	1 hour
NEXT WEEK	09	Scout Request Card Cleanup	High	2 hours
NEXT WEEK	10	Notification Card Removal	High	2 hours
NEXT WEEK	12	Alternate Position Tap Fix	High	2–3 hours
NEXT WEEK	19	Coach-First Data Entry Flow	High	1–2 days
AFTER ABOVE	13	Club Trial Feature	Medium	3–5 days
AFTER ABOVE	11	Marketplace Shortcut Removal	Medium	30 minutes
AFTER ABOVE	18	Data Portability on Club Change	High	1–2 days

Built in Kenya. Built for Africa. Ready for the World.

Talent Graph — Verve Vigor Limited | Stanley Omenda | May 2026 | Confidential